

About Me

Hi, I'm Alexander Hennig, an Indie Game Developer and Narrative Designer from Germany.

This portfolio focuses on Ruins of Blackspire and walks through my Design, the systems I built from scratch, and some of the challenges I faced along the way.

I'm currently working as a Narrative Designer at Gamucatex on Tectonicus a historical, card-based auto-battler strategy game.

In my free time I work on my game, Ruins of Blackspire a 2D strategy dungeon crawler focused on tactical combat and resource management.

My preferred engine is Godot, which I used to develop Ruins of Blackspire. I also bring extensive experience in Unity through prior projects and my current position at Gamucatex. While Godot utilizes its own language, GDScript which I mastered during the development of Ruins of Blackspire, I am also well-versed in object-oriented programming languages like C++ and Python through various personal projects.

I also have a lot of knowledge in Media Design Concepts and using Adobe Programs from my Degree as a Media Technical Assistant but after that I put my focus onto Coding and Game Design.

I really like RTS games, hence why Ruins of Blackspire has many mechanics related to them. It's nice to be able to choose how you want to approach any given situation with the tools you have available, which is a feeling I really like to replicate in my game. But I also really like RPGs and shooters which influenced a lot of my Story Design and how to create compelling Combat encounters.

If you are interested you can reach out via E-Mail or LinkedIn.

E-Mail: AlexanderHennigBusiness@outlook.de

LinkedIn: <http://linkedin.com/in/alexander-hennig-309b00223>

Ruins of Blackspire Demo is available on:

Steam: https://store.steampowered.com/app/3714780/Ruins_of_Blackspire_Demo/

Itch.io: <https://pixelbrew.itch.io/ruins-of-blackspire>

Ruins of Blackspire

Ruins of Blackspire is a top-down, 2D, Dungeon Crawler with RTS inspired gameplay. You pick a class and go through a dungeon full of enemies, items and abilities.

Genre: RTS, Dungeon Crawler, Rogue-like

Engine: Godot

Duration: 2023 - 2026

Platform: PC (Steam)



Roles and responsibilities:

Game Design:

- Designed the Game to support full mouse-only control, allowing players to play without a keyboard
- Developed slow-paced, RTS-inspired combat to emphasize tactical decision-making.
- Balanced unit design around high health and low damage to encourage deliberate resource management.

Programming:

- Created a scalable inventory system that supports equippable items, consumables, and spells.
- Made a system that allows easy creation and implementation of new items/spells.
- Focused on strong internal communication between inventory systems for features like cooldown tracking and spell slot behavior.
- Created unique solution to allow for RTS like pathfinding without avoidance.

Art & Sound:

- Directed and coordinated with contributing artists, providing feedback and guidance to match the tone and needs of the game.
- Implemented all Art and Sound Assets into the Game Engine

Marketing & Publishing:

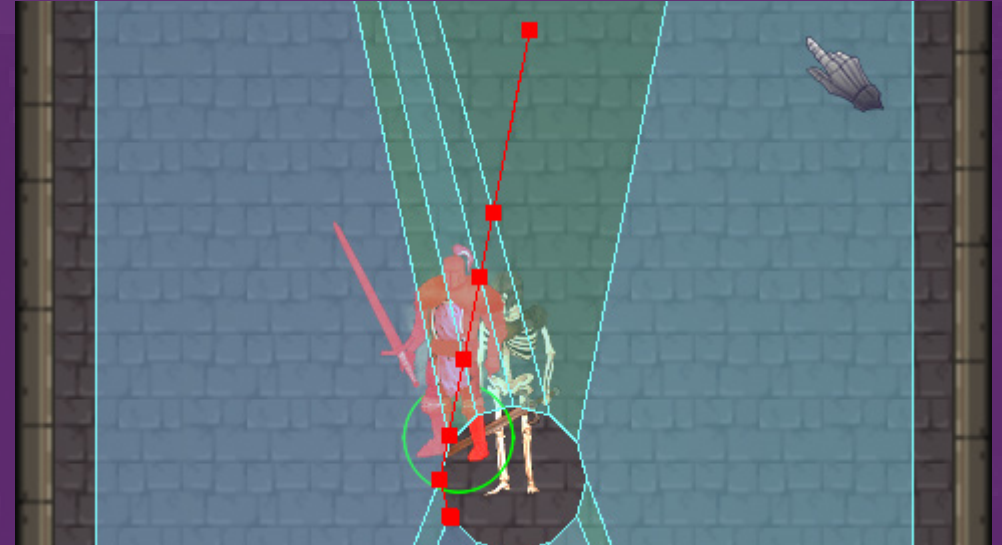
- Managed the entire social media presence of the game.
- Handled all tasks related to Steam publishing: store page, trailer, tags, and everything else.
- Almost daily updates to the game published on GitHub

Design

Pathfinding

For a game inspired by RTS mechanics, responsive and reliable pathfinding is important to the player experience. From the start, I knew this would be one of the most time-consuming systems to research and implement, and it ended up being even more time consuming and difficult than I could ever have anticipated.

A lot of development time was spent iterating heavily on this system, fixing issues, testing interactions, and making it feel and act just the way I had planned from the very beginning. The system I have in the game right now allows units to path around others and navigate around obstacles just as planned, but I'm still testing performance impacts to see if I can give it more precision.



Explanation

The reason I planned to go with this approach is because of my experience playing RTS games. Especially in Warcraft 3, I have seen a lot of discussions about why the pathfinding feels so great compared to newer RTS games that try to make pathing more flawless and fluent.

Since it is unlikely that I can create pathfinding logic by myself that a game studio with a high budget took years to make, I decided to go with simpler logic. Now, after all the time iterating, it seems like it was a good call.

Implementation

Pathfinding

As mentioned earlier, pathfinding was by far the most time-consuming feature in the game. After extensive testing, I ended up implementing a custom solution because Godot did not natively support the behavior I needed.

The standard approach would be to use a `NavigationAgent2D` with avoidance enabled, allowing units to path around each other. However, this did not work reliably for my use case. Instead, I created a system where every unit has its own dynamic collision area (referred to as a “Proxy”). This area is moved as a child of the relevant map segment during gameplay.

The reason for this workaround is that areas can only be subtracted from a navmesh if they are direct children of a `NavigationRegion2D`. Given my map structure, this wasn’t feasible. So whenever a unit enters a new map segment, its Proxy area is reparented to that segment. The segment then rebakes (recalculates) its navmesh, effectively carving out the occupied space. This prevents other units from pathing through it, enabling true dynamic path blocking.

Explanation

I decided to go with this approach in the end partially because of what I mentioned before, but also because I really enjoyed the unit blocking mechanics in Warcraft 3 and DOTA 2. Being able to physically block enemy escape routes with your own units adds a deep, skill-based layer to gameplay that I wanted to replicate.

Of course, for this mechanic to be meaningful, the game environment must support it. If units simply charge straight at the player and attack, path blocking rarely comes into play in a way the player can meaningfully interact with. My design intentionally creates scenarios where positioning and body blocking is something the player will (hopefully) regularly interact with.

Code Examples

Pathfinding

The way I have it set up is that the nav_blocker_proxy, which is a child of my player and enemies, is sent to the map segment that the specific unit entered. This is then used to carve a hole into the navmesh whenever the timer times out.

The numbers are still something I'm playing with, but the core of this so far in my testing has not caused any issues. However, the proxy is much newer than the rest, so I have not yet tested every possible scenario.

player.gd

```
1408 >|
1409 >|
1410 ▾ func navmeshBlock(status: bool):
1411 ▾ >| if nav_blocker_proxy and nav_blocker_proxy.has_node("CollisionShape2D"):
1412 >| >| nav_blocker_proxy.get_node("CollisionShape2D").disabled = status
```

levelSegmenttest.gd

```
7 @onready var player_carver = Area2D
8 var rebake_timer
9
10 ▾ func _ready():
11 >| rebake_timer = $rebake_timer
12 >| >|
13 ▾ func _on_area_2d_body_entered(body):
14 ▾ >| if body.is_in_group("units"):
15 >| >| start_rebake_timer()
16 >| >| var proxy = body.nav_blocker_proxy
17 ▾ >| >| if proxy:
18 >| >| >| proxy.call_deferred("reparent", nav_region)
19
20
21 ▾ func _on_area_2d_body_exited(body):
22 ▾ >| if body.is_in_group("units"):
23 >| >| stop_rebake_timer()
24 >|
25 ▾ func start_rebake_timer():
26 >| rebake_timer.start()
27 |
28 ▾ func stop_rebake_timer():
29 >| rebake_timer.stop()
30 >|
31 ▾ func get_navigation_region() -> NavigationRegion2D:
32 >| return $NavigationRegion2D
33 >|
34 ▾ func _on_timer_timeout():
35 >| nav_region.bake_navigation_polygon()
```

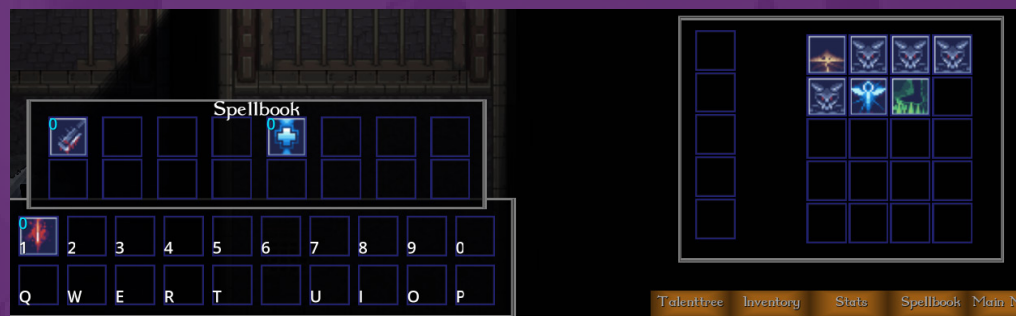

Design

Item & Spell Inventory System

The original plan was for the player to have 20+ spell slots that can all have a spell attached to them. So I needed a scalable inventory system that allows me to adjust it based on how many spells a player could access.

One important pillar for the game is that you have total freedom on how to approach any combat encounter. One way to achieve this is to give the player a bunch of spells and enough slots to use any of them at any time. This requires a lot of spells to be designed which, in turn, can break a lot of other features.

Since both inventories used the same code, I was able to easily add things like drag-and-drop or other QoL features to both and not worry about errors through missing dependencies for example, when moving something from one inventory to the other.



Explanation

This system is inspired by MMO spell systems that allow you to have a wide variety of spells for many different situations. Since I planned very early on that the core gameplay loop is supposed to be a sandbox in terms of how you want to approach enemies or enemy groups, and because I played a good amount of MMOs (mostly WoW), this seemed like a good fit based on my experience using those systems in general.

Because this system goes very deep, especially in terms of how many QoL features you can add, it is not entirely done. However, all base functionality works decently, although there is still a lot of design work that needs to be done for the remaining required features, which then need to be coded as well.

Implementation

Item & Spell Inventory System

I started creating the spell inventory and my spell casting mechanics, and after I was happy with the base functionality, I copied it for my item inventory. This worked pretty well, outside of having to fix missing types manually. For example, dragging an item into a spell bar obviously works, but because of the type mismatch the game would immediately crash.

After this was done, most of the work on this system was implementing the more important functionality and fixing bugs they caused. The biggest one I made so far was a drag-and-drop from one spell slot to another, and also for later planned functionality a consumable item from the inventory onto the spell bars so they are usable like spells.

The whole inventory system required many interconnected scripts over multiple scenes and signals and is likely the most complex system I have made so far. But because of that, it was also the most interesting to work on, since it is very unique and complex but also very core to the gameplay loop.

Explanation

Obviously for a 2D Dungeon Crawler game this is not a spell system you would usually find. But simply because when I was figuring out how it works I realized it is fairly simple to do and is an interesting approach that, as far as I'm aware, is not widely used. So I decided, why not?

For the inventory, if I wanted something different like old-school inventory management from something like Diablo 2, I can still change it. But for the spell inventory part, it ticks all the boxes I need for what I planned, what spells to bring to the game, and the connection to the inventory also plays an important part.

Code Examples

As mentioned before, both inventory systems are fairly complex and spread over multiple scripts, with probably something like 500+ lines of code across at least 5 scripts. These 3 screenshots help explain what is happening.

The inventory is updated in real time with cooldowns etc. using the „spellbarUI.gd“ script (this was very time-consuming to set up). When the player activates a spell, the „spellbar_ui_slot.gd“ sends a signal from that slot with the spell to the player. The player then checks if the spell is available and if the player has the resources for it. If true, it instantiates the spell as a new scene which has its own functionality.

```
spellbar_ui_slot.gd(*) Online Docs

75 >| >| reset_cooldown_visual()
76 >| >|
77 # Resets cooldown visual to its default state
78 func reset_cooldown_visual():
79 >|
80 >| time.text = ""
81 >| cooldown.value = 0
82 >| set_process(false)
83 >| spellbutton.disabled = false
84 >| >|
85 >| print("cooldown stopped")
86 >|
87 >|
88 # Called when cooldown needs to be updated manually (e.g., when moving spell)
89 func update_cooldown_visual(spell_id, remaining_time, cooldown_time):
90 >|
91 if spell_id != null and item_scene != null:
92 >| >|
93 if item_scene.spell_ID == spell_id:
94 >| >| >| time.visible = true
95 >| >| >| cooldown.max_value = cooldown_time
96 >| >| >| cooldown.value = remaining_time
97 >| >| >| set_process(true)
98 >| >| >|
```

```
spellbarUI.gd Online Docs

9 >|
10 func update_keys():
11 for i in range(slots.size()):
12 >| >| var events = InputMap.action_get_events("Spellslot%d" % i)
13 if events.size() > 0 and events[0] is InputEventKey:
14 >| >| >| var key_event = events[0]
15 >| >| >| var key_text = ""
16 >| >| >|
17 >| >| >| # Add modifier prefix if needed
18 if key_event.shift_pressed:
19 >| >| >| >| key_text += "Shift + "
20 if key_event.alt_pressed:
21 >| >| >| >| key_text += "Alt + "
22 if key_event.ctrl_pressed:
23 >| >| >| >| key_text += "Ctrl + "
24 if key_event.meta_pressed:
25 >| >| >| >| key_text += "Meta + " # for mac
26
27 >| >| >| key_text += OS.get_keycode_string(key_event.keycode)
28 >| >| >| slots[i].hotkey.text = key_text
29 >| >| >|
```

```
player.gd(*) Online Docs Search Help

632 var spell_scene
633 var casted_spell
634 func castSpell(spell):
635 >| AudioManager.play_sound(AudioManager.casting, global_position)
636 >|
637 spell_scene = spell.instantiate()
638 >| spell_scene.global_position = self.global_position
639 >| spell_scene.set_destination(target_position)
640 >| spell_scene.set_caster(self)
641 >| var spell_resource = spell_scene.spell_resource_types.keys()[spell_scene.spell_resource]
642 >|
643 >| >| >|
644 if casting == false:
645 >| >| casted_spell = spell_scene
646 >| >|
647 if spell_scene.spellproperties.mana_cost > current_mana or spell_resource != player_resource and spell_scene.spellproperties.mana_cost != 0:
648 >| >| >| abort_cast.emit(casted_spell)
649 elif spell_scene.spellproperties.mana_cost > current_rage or spell_resource != player_resource and spell_scene.spellproperties.mana_cost != 0:
650 >| >| >| abort_cast.emit(casted_spell)
651 >| >|
652 else:
653 >| >| >| if spell_scene.spellproperties.cast_time == 0:
654 >| >| >| >| cast_time.wait_time = 0.0001
655 >| >| >| else:
656 >| >| >| >| cast_time.wait_time = casted_spell.spellproperties.cast_time
657 >| >| >|
658 >| >| >| >| cast_time.start()
659 >| >| >| >| cast_bar.visible = true
660 >| >| >| >| casting = true
661 >| >| >| >| change_state(Playerstates.CASTING)
662 >| >| >|
663 else:
664 >| >| >| if spell_scene != casted_spell:
665 >| >| >| >| abort_cast.emit(spell_scene)
666 >| >| >| >| print("castspell failed")
```